

Análise Comparativa de Algoritmos de Compressão de Arquivos

LUCAS SOLLER, Universidade Federal do Rio Grande do Sul, Brasil

LUCAS NOGUEIRA, Universidade Federal do Rio Grande do Sul, Brasil

Este trabalho apresenta uma análise comparativa de desempenho de quatro algoritmos de compressão de arquivos amplamente utilizados: gzip, bzip2, xz e zstd. Utilizando medições em ambiente real com containerização Docker para garantir reprodutibilidade, avaliamos métricas como tempo de compressão, tempo de descompressão e taxa de compressão. Os experimentos foram conduzidos com arquivos de texto gerados através de Cadeias de Markov, em tamanhos de 10MB, 100MB e 1000MB. Os resultados demonstram os trade-offs entre velocidade e eficiência de compressão, fornecendo insights práticos para a escolha do algoritmo mais adequado em diferentes cenários de uso.

Additional Key Words and Phrases: compressão de arquivos, análise de desempenho, gzip, bzip2, xz, zstd, benchmarking

1 Introdução

A compressão de arquivos é uma tarefa rotineira e fundamental no ambiente de desenvolvimento de software, administração de sistemas e ciência de dados. Com o crescimento exponencial do volume de dados gerados e armazenados, a escolha do algoritmo de compressão adequado torna-se cada vez mais crítica para otimizar espaço de armazenamento e transferência de dados em redes.

Este trabalho tem como objetivo analisar comparativamente quatro das ferramentas de compressão mais utilizadas em sistemas Unix-like: **gzip**, **bzip2**, **xz** e **zstd**. Cada uma dessas ferramentas implementa algoritmos distintos com características próprias de desempenho e eficiência.

A escolha deste objeto de estudo baseia-se na relevância prática do tema e na vivência do grupo. Compreender os *trade-offs* entre os diferentes algoritmos é crucial para tomar decisões informadas sobre qual ferramenta utilizar em cenários específicos, como backup de dados, distribuição de pacotes de software ou otimização de largura de banda em redes.

2 Fundamentação Teórica

Nesta seção, descrevemos brevemente os algoritmos subjacentes a cada uma das ferramentas de compressão analisadas.

2.1 gzip (DEFLATE)

O gzip [2] é baseado no algoritmo DEFLATE, que combina o algoritmo LZ77 (Lempel-Ziv 1977) com codificação Huffman. É amplamente utilizado devido à sua alta velocidade de compressão e descompressão, embora ofereça uma taxa de compressão moderada em comparação com algoritmos mais recentes.

2.2 bzip2 (Burrows-Wheeler)

O bzip2 [5] utiliza a transformada de Burrows-Wheeler (BWT) seguida de codificação Move-to-Front e codificação Huffman. Este processo resulta em taxas de compressão superiores ao gzip, porém com maior tempo de processamento, especialmente na descompressão.

2.3 xz (LZMA2)

O xz [6] é baseado no algoritmo LZMA2 (Lempel-Ziv-Markov chain Algorithm), que oferece as maiores taxas de compressão entre os quatro algoritmos analisados. No entanto, isso vem com o

custo de tempos de processamento significativamente mais longos, tanto para compressão quanto para descompressão.

2.4 zstd (Zstandard)

O zstd [1], desenvolvido pelo Facebook, é um algoritmo moderno que busca equilibrar alta velocidade (comparável ao gzip) com excelente taxa de compressão (competindo com xz). Utiliza uma combinação de técnicas incluindo correspondência de dicionário, codificação de entropia FSE (Finite State Entropy) e oferece níveis de compressão configuráveis.

3 Metodologia

O método de análise escolhido é a medição em ambiente real. Este método permite a obtenção de dados concretos e reproduzíveis do desempenho de cada algoritmo em condições operacionais típicas.

3.1 Ambiente Experimental

Para garantir a reprodutibilidade e isolamento dos testes, foi utilizado Docker. Isso permite que o ambiente de execução seja consistente, independentemente da máquina em que o projeto for executado, eliminando variações causadas por diferentes sistemas operacionais ou versões de pacotes.

Configurações do Host:

- Sistema Operacional: Ubuntu 22.04.5 LTS x86_64
- Hardware: Dell Inspiron 3584
- CPU: Intel i3-7020U (4 threads) @ 2.300GHz
- RAM: 4 GB

Imagem Docker: debian:bookworm-slim com os seguintes pacotes:

- bzip2=1.0.8-5+b1
- gzip=1.12-1
- xz-utils=5.4.1-1
- zstd=1.5.4+dfsg2-5
- sysstat=12.6.1-1 (iostat)
- procps=2:4.0.2-3 (vmstat)
- linux-perf=6.1.153-1

3.2 Ferramentas de Instrumentação

perf Ferramenta avançada para profiling de desempenho [4] que permite coletar dados detalhados sobre a atividade da CPU, como cache misses, instruções executadas e ciclos de CPU. Foi utilizado `perf stat` para obter estatísticas resumidas.

iostat Ferramenta para monitorar a atividade de E/S do sistema (leitura/escrita em disco). Essencial para entender o impacto dos algoritmos nos discos.

vmstat Ferramenta para monitorar a memória virtual. Útil para entender o uso de memória durante a compressão e descompressão de arquivos grandes.

Scripts Bash Para automatizar a execução de múltiplas rodadas de testes e a coleta de dados, garantindo a consistência dos experimentos.

3.3 Geração dos Dados de Teste

Para garantir a aleatoriedade e generalidade dos resultados, os arquivos de teste não são fixos, mas gerados aleatoriamente utilizando o algoritmo de Cadeias de Markov, conforme descrito por Kernighan e Pike [3].

Na implementação, foi utilizado um arquivo base (“A Tale of Two Cities”) para treinar o modelo de Markov, e o tamanho do arquivo final foi especificado em MB. Este método garante que os testes simulem a compressão de dados textuais realistas, sem padrões artificiais que possam favorecer ou prejudicar um algoritmo específico.

Os tamanhos de arquivo utilizados foram: 10MB, 100MB e 1000MB.

3.4 Procedimento Experimental

O procedimento de benchmark segue os seguintes passos para cada tamanho de arquivo:

- (1) **Geração do arquivo de teste:** Um arquivo de texto é gerado utilizando o gerador de Cadeias de Markov com o tamanho especificado.
- (2) **Para cada algoritmo (gzip, bzip2, xz, zstd):**
 - (a) Para cada repetição (3 execuções):
 - (i) Limpar o cache de páginas do sistema: `sync && echo 3 > /proc/sys/vm/drop_caches`
 - (ii) Iniciar monitoramento com `vmstat` e `iostat`
 - (iii) Executar compressão com `perf stat` para coletar métricas de hardware
 - (iv) Medir tamanho do arquivo comprimido
 - (v) Limpar cache novamente
 - (vi) Executar descompressão com `perf stat`
 - (vii) Salvar resultados em arquivo CSV
 - (viii) Aguardar 10 segundos (cooldown)

Os eventos de hardware coletados pelo `perf` incluem: task-clock, context-switches, cpu-migrations, page-faults, cycles, instructions, branches e branch-misses.

3.5 Métricas Coletadas

Tempo de Compressão O tempo total (task-clock em ms) que cada ferramenta leva para compactar um arquivo.

Tempo de Descompressão O tempo que cada ferramenta leva para descompactar o arquivo gerado.

Taxa de Compressão Calculada pela fórmula: $\frac{\text{Tamanho comprimido}}{\text{Tamanho original}} \times 100$. Valores menores indicam maior eficiência.

Ciclos de CPU Número total de ciclos de processador utilizados.

Instruções Executadas Número de instruções de máquina executadas.

Branch Misses Número de predições de branch incorretas, indicando a complexidade do fluxo de controle do algoritmo.

3.6 Reprodução dos Experimentos

O projeto está organizado em duas fases distintas, cada uma com metodologia e estrutura próprias.

Pré-requisitos:

- Docker instalado e em execução
- Permissões para executar containers privilegiados (necessário para `perf`)

Fase 2 (arquivos de 10MB, 100MB e 1000MB):

`cd phase2`

```
./run_experiments.sh
```

Nesta fase, para cada tamanho de arquivo e cada repetição, um **novo arquivo de teste é gerado** utilizando o gerador de Cadeias de Markov. O script compila o gerador (`markov_chain.cpp`), gera o arquivo, executa a compressão/descompressão e repete o processo. Isso introduz variabilidade nos dados de teste entre execuções.

Fase 3 (arquivos de 1MB a 25MB, metodologia refinada):

```
cd phase3
./run_experiments.sh
```

Nesta fase, os arquivos de teste foram **gerados previamente uma única vez** e empacotados em `files.tar.xz`. O script extrai os arquivos e executa todos os experimentos sobre os mesmos dados, garantindo que as comparações entre algoritmos sejam feitas em condições idênticas. Esta abordagem elimina a variabilidade da geração de dados e permite análises estatísticas mais precisas.

Diferenças principais entre as fases:

- **Fase 2:** Geração dinâmica de arquivos a cada experimento; tamanhos grandes (até 1GB); foco em escalabilidade.
- **Fase 3:** Arquivos pré-gerados e reutilizados; granularidade fina (1-25MB); foco em precisão estatística.

Os notebooks Jupyter (`graphs_phase2.ipynb` e `graphs_phase3.ipynb`) podem ser utilizados para gerar os gráficos a partir dos dados coletados.

4 Resultados

Esta seção apresenta os resultados obtidos nos experimentos, divididos em duas fases de análise.

4.1 Resultados da Fase 2

Na segunda fase do projeto, foram realizados 3 experimentos para cada um dos quatro algoritmos, utilizando arquivos de texto de 10MB, 100MB e 1000MB. Nesta etapa, um novo arquivo de texto era gerado para cada experimento.

As Figuras 1, 2 e 3 apresentam os resultados para arquivos de 10MB, 100MB e 1000MB, respectivamente. Cada gráfico mostra três métricas: tempo de compressão (ms), tempo de descompressão (ms) e taxa de compressão (%).

Observações da Fase 2:

- O algoritmo XZ apresenta o maior tempo de compressão entre todos os algoritmos.
- O algoritmo BZIP2 apresenta o maior tempo de descompressão.
- Nesta etapa, a métrica de taxa de compressão estava invertida (quanto maior, pior). Isso foi corrigido na fase seguinte.
- Com quase 40% de taxa, o GZIP possui o pior desempenho em eficiência de compressão, ou seja, 40% do tamanho original não foi comprimido. O XZ apresentou apenas 10% do tamanho original não comprimido.

4.2 Resultados da Fase 3

Na terceira fase, a metodologia foi refinada para garantir maior rigor científico. As principais mudanças foram:

- **Arquivos pré-gerados:** Todos os arquivos de teste (de 1MB a 25MB) foram gerados uma única vez usando Cadeias de Markov e armazenados em `files.tar.xz`. Os mesmos arquivos são reutilizados em todas as execuções, eliminando a variabilidade da Fase 2.

- **Preload em cache:** Antes de cada bateria de testes, o arquivo é pré-carregado na memória com `cat arquivo > /dev/null`, eliminando o efeito de *cold start* e garantindo que as medições reflitam apenas o tempo de processamento do algoritmo, sem interferência de I/O de disco.
- **Granularidade fina:** Em vez de apenas 3 tamanhos (10, 100, 1000MB), foram utilizados 25 tamanhos diferentes (1MB a 25MB), permitindo análise mais detalhada do comportamento dos algoritmos.
- **Métricas expandidas:** Além das métricas básicas, foram coletados dados de cache misses, page faults e chamadas de sistema de I/O.

Foram realizadas 3 execuções para cada combinação de algoritmo e tamanho de arquivo, totalizando 300 experimentos (4 algoritmos × 25 tamanhos × 3 repetições).

4.2.1 Tempo de Compressão. A Figura 4 apresenta o tempo de compressão em função do tamanho do arquivo. O algoritmo **xz** apresenta o maior tempo de compressão, crescendo linearmente de aproximadamente 0,6s para 1MB até 21,5s para 25MB. Em contraste, o **zstd** mantém tempos extremamente baixos (0,01s a 0,26s), sendo até 80 vezes mais rápido que o xz para arquivos grandes. A Figura 5 apresenta os mesmos dados com regressão linear e projeção para arquivos de até 35MB, permitindo estimar o comportamento para arquivos maiores.

4.2.2 Tempo de Descompressão. A Figura 6 mostra o tempo de descompressão. Diferentemente da compressão, o **bzip2** apresenta os piores tempos de descompressão, chegando a 1,7s para arquivos de 25MB. Os demais algoritmos (gzip, xz e zstd) mantêm tempos de descompressão similares e baixos, com o zstd novamente liderando com tempos inferiores a 0,1s.

A Figura 7 apresenta os mesmos dados com regressão linear e projeção para arquivos maiores.

4.2.3 Taxa de Compressão. A Figura 8 apresenta a taxa de compressão (quanto maior, melhor). O **xz** atinge as melhores taxas, comprimindo até 90% do arquivo original para tamanhos maiores. O **gzip** apresenta desempenho consistentemente inferior, mantendo-se em torno de 63% de compressão independentemente do tamanho do arquivo.

A Figura 9 apresenta a taxa de compressão com regressão SPLINE e projeção para arquivos maiores.

4.2.4 Análise com Intervalo de Confiança. As Figuras 10 e 11 apresentam os tempos de compressão com intervalo de confiança de 95%, demonstrando a estabilidade das medições. As barras de erro são praticamente invisíveis, indicando alta reprodutibilidade dos experimentos.

5 **Discussão**

5.1 **Resumo Comparativo dos Algoritmos**

A Tabela 1 apresenta uma classificação qualitativa dos algoritmos nas principais métricas avaliadas.

Table 1. Classificação dos algoritmos por métrica de desempenho.

Métrica	gzip	bzip2	xz	zstd
Velocidade de Compressão	Bom	Regular	Ruim	Excelente
Velocidade de Descompressão	Bom	Ruim	Bom	Excelente
Taxa de Compressão	Regular	Bom	Excelente	Bom
Uso de CPU	Baixo	Alto	Muito Alto	Muito Baixo
Avaliação Geral	Bom	Regular	Especializado	Excelente

A Tabela 2 apresenta os valores médios e desvio padrão obtidos considerando todos os tamanhos de arquivo (1MB a 25MB), totalizando 75 amostras por algoritmo.

Table 2. Valores médios e desvio padrão para arquivos de 1–25MB (75 amostras por algoritmo).

Algoritmo	Compressão (ms)	Descompressão (ms)	Taxa (%)
gzip	1448 ± 805	130 ± 73	37,35 ± 0,04
bzip2	1734 ± 964	869 ± 487	21,68 ± 0,28
xz	10975 ± 6376	117 ± 57	11,88 ± 2,63
zstd	141 ± 74	39 ± 19	24,05 ± 1,15

5.2 Análise dos Trade-offs

Os resultados obtidos confirmam os trade-offs conhecidos entre os algoritmos de compressão:

gzip oferece um equilíbrio razoável entre velocidade e simplicidade, sendo ideal para cenários onde compatibilidade é essencial. Entretanto, sua taxa de compressão é a mais baixa entre os algoritmos testados.

bzip2 apresenta melhor taxa de compressão que o gzip, mas com penalidade significativa no tempo de descompressão (até 7x mais lento que gzip). É adequado para arquivamento de longo prazo onde a descompressão é infrequente.

xz oferece as melhores taxas de compressão (redução de até 90%), mas com os maiores tempos de processamento. Recomendado para distribuição de software e arquivos que serão descomprimidos muitas vezes após uma única compressão.

zstd demonstra ser a alternativa mais moderna e versátil, oferecendo velocidades até 80x superiores ao xz com taxas de compressão competitivas. É particularmente adequado para aplicações em tempo real, backups incrementais e streaming de dados.

6 Conclusão

Este trabalho apresentou uma análise comparativa detalhada de quatro algoritmos de compressão de arquivos: gzip, bzip2, xz e zstd. Através de experimentos controlados em ambiente Docker, foram coletadas métricas de tempo de compressão, descompressão e taxa de compressão para arquivos de texto de 1MB a 25MB.

Os resultados demonstram que não existe um algoritmo universalmente superior, mas sim diferentes trade-offs que devem ser considerados conforme o caso de uso específico. Com base nos experimentos realizados, recomendamos:

- **Para velocidade máxima:** zstd — até 80x mais rápido que xz na compressão e 24x mais rápido que bzip2 na descompressão.
- **Para melhor taxa de compressão:** xz — atinge taxas de até 90%, reduzindo arquivos de 25MB para apenas 2,5MB.
- **Para uso geral moderno:** zstd — combina alta velocidade com boa taxa de compressão, sendo a escolha mais equilibrada.
- **Para compatibilidade legado:** gzip — amplamente suportado em todos os sistemas Unix-like.
- **Evitar para descompressão frequente:** bzip2 — seu tempo de descompressão é significativamente maior que os demais.

O algoritmo zstd destaca-se como a melhor escolha para a maioria dos cenários modernos, oferecendo o melhor equilíbrio entre velocidade e eficiência de compressão. O xz permanece relevante para casos onde o tamanho final é crítico e o tempo de compressão não é uma restrição.

Como trabalhos futuros, sugerimos:

- Análise com diferentes tipos de arquivos (binários, imagens, dados científicos)
- Avaliação de diferentes níveis de compressão de cada algoritmo
- Medição do consumo de memória durante a compressão
- Análise de compressão paralela (multi-threaded)

References

- [1] Facebook. [n. d.]. Zstandard - Fast real-time compression algorithm. <https://facebook.github.io/zstd/>. Acessado em: 2025.
- [2] GNU Project. [n. d.]. gzip - GNU Zip. <https://www.gnu.org/software/gzip/>. Acessado em: 2025.
- [3] Brian W. Kernighan and Rob Pike. 1999. *The Practice of Programming*. Addison-Wesley, Reading, Massachusetts.
- [4] Linux Kernel Organization. [n. d.]. perf: Linux profiling with performance counters. <https://perf.wiki.kernel.org/>. Acessado em: 2025.
- [5] Julian Seward. [n. d.]. bzip2 and libbzip2. <https://sourceware.org/bzip2/>. Acessado em: 2025.
- [6] Tukaani Project. [n. d.]. XZ Utils. <https://tukaani.org/xz/>. Acessado em: 2025.

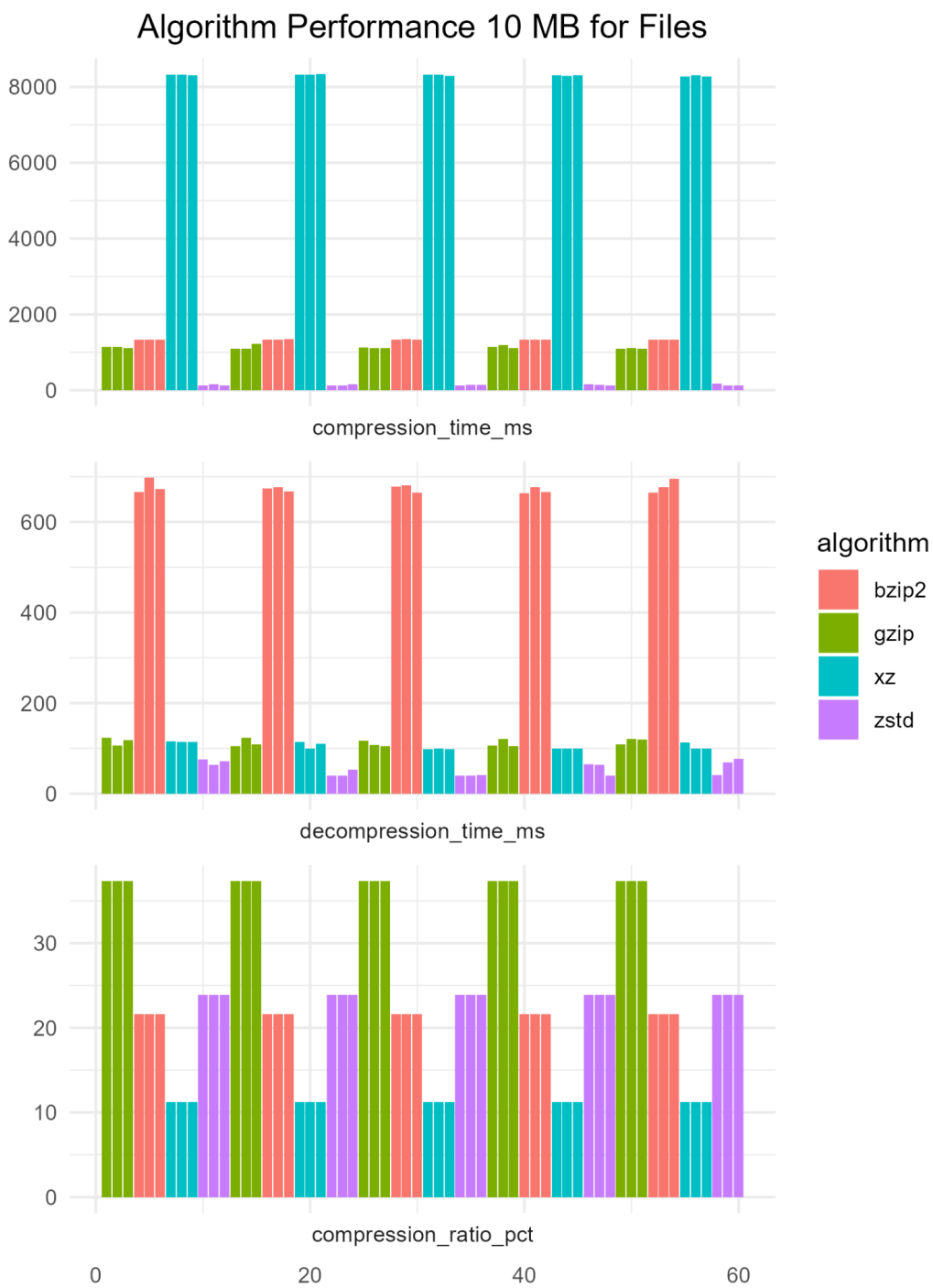


Fig. 1. Desempenho dos algoritmos com arquivos de 10MB: tempo de compressão, descompressão e taxa de compressão.



Fig. 2. Desempenho dos algoritmos com arquivos de 100MB.



Fig. 3. Desempenho dos algoritmos com arquivos de 1000MB.

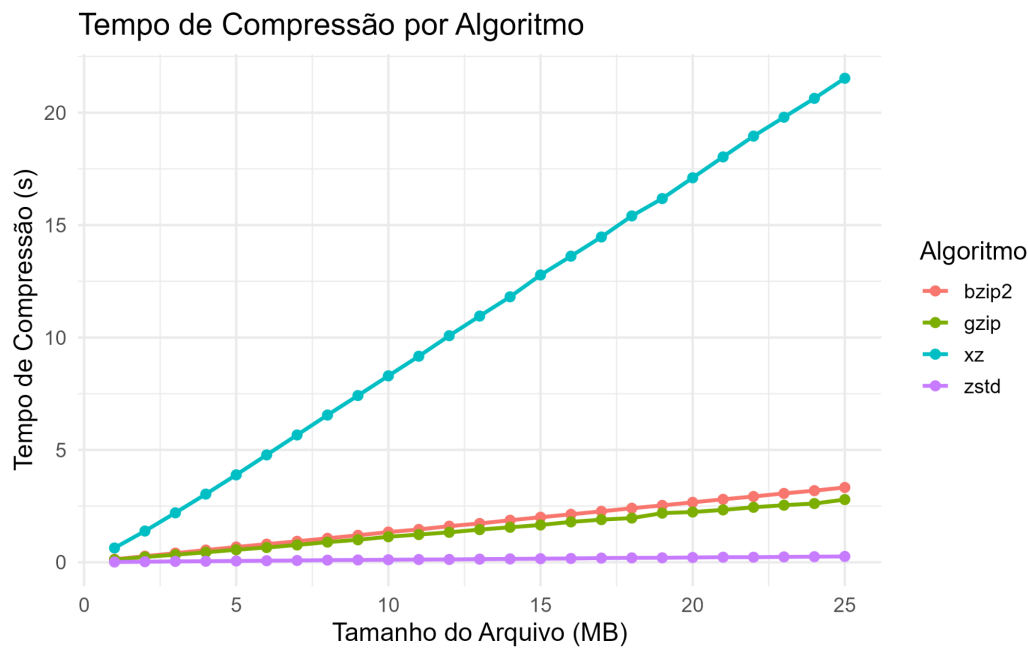


Fig. 4. Tempo de compressão por algoritmo em função do tamanho do arquivo.

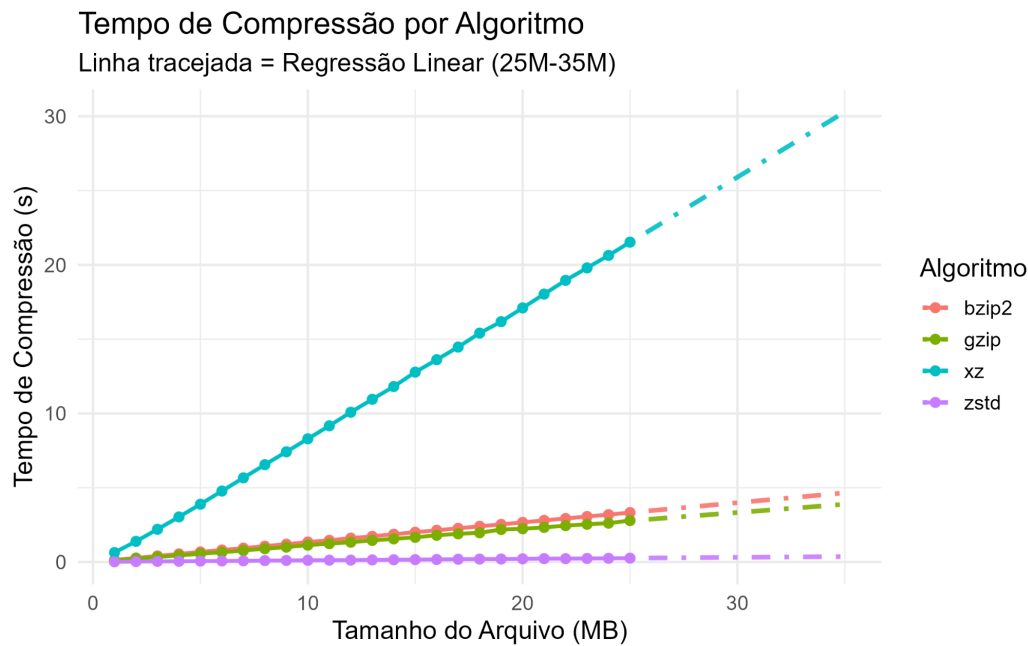


Fig. 5. Tempo de compressão com regressão linear (projeção 25M-35M).

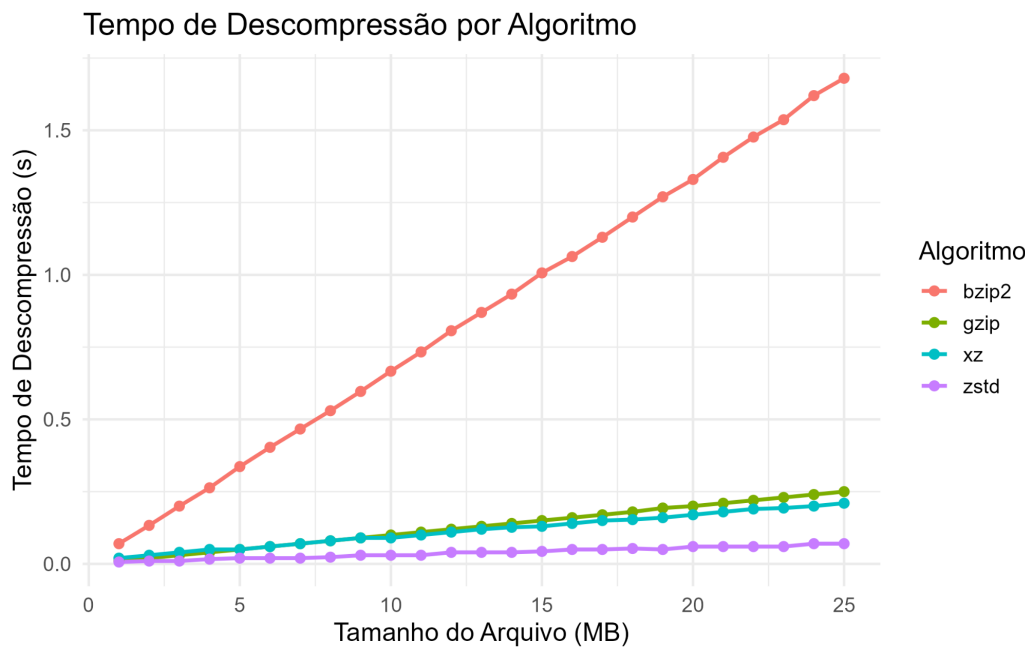


Fig. 6. Tempo de descompressão por algoritmo em função do tamanho do arquivo.

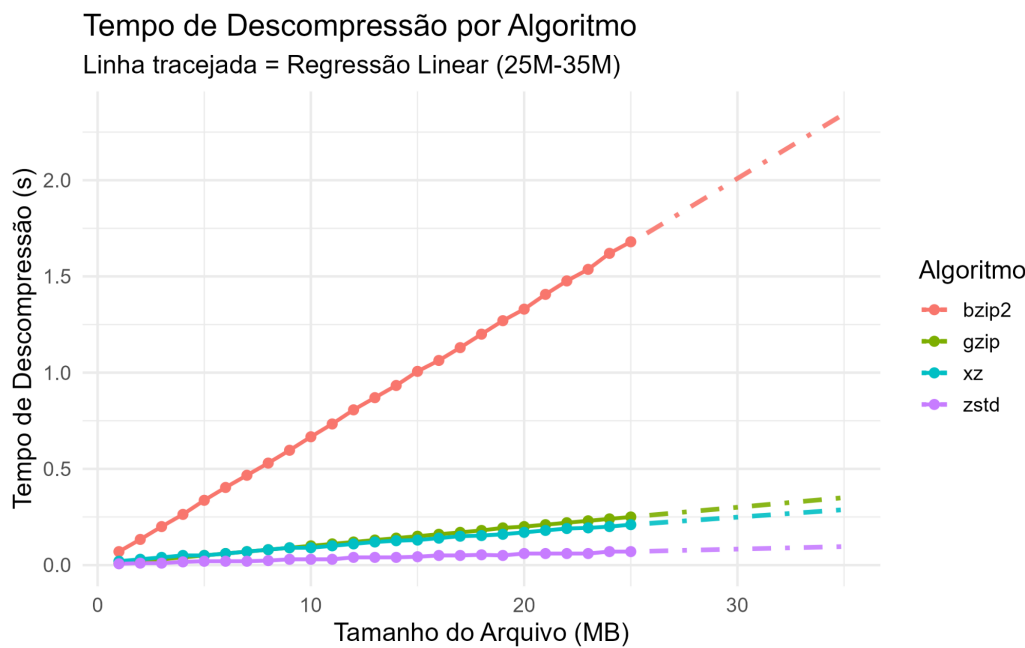


Fig. 7. Tempo de descompressão com regressão linear (projeção 25M-35M).

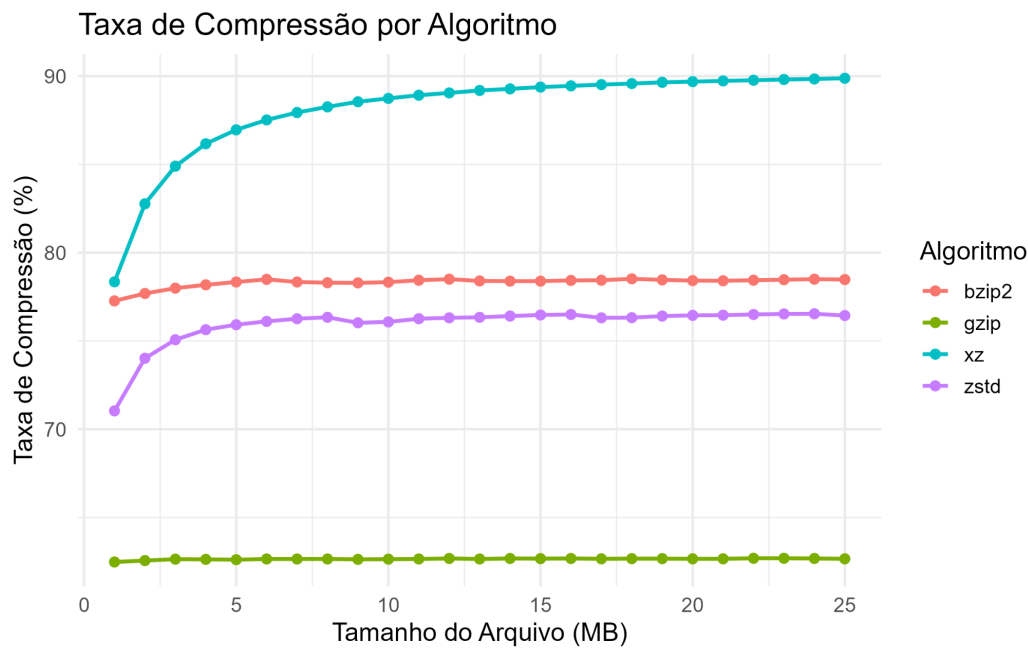


Fig. 8. Taxa de compressão por algoritmo em função do tamanho do arquivo.

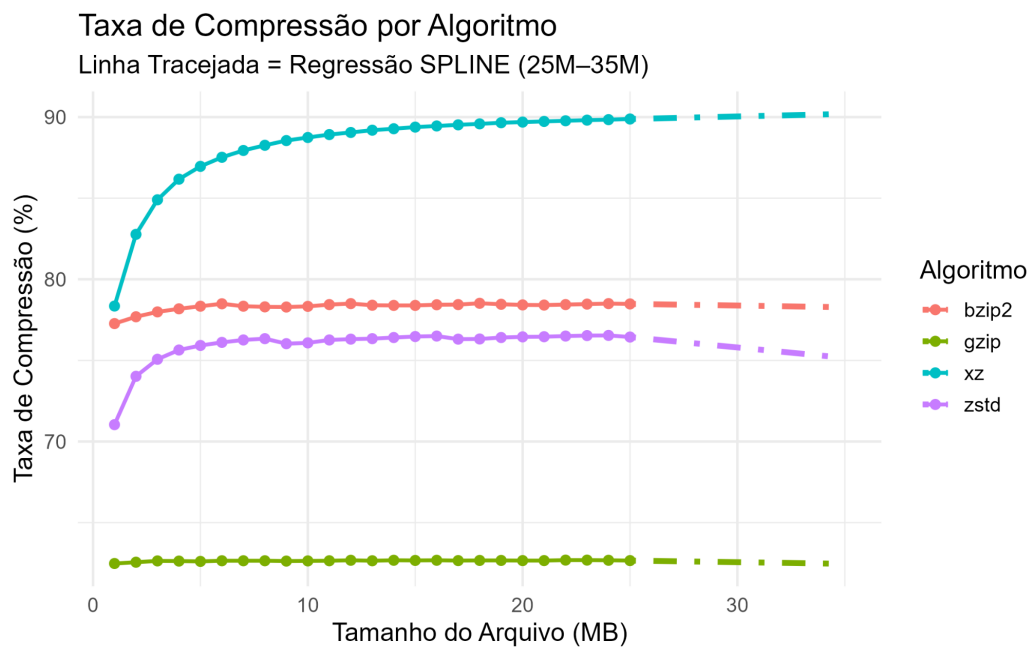


Fig. 9. Taxa de compressão com regressão SPLINE (projeção 25M-35M).

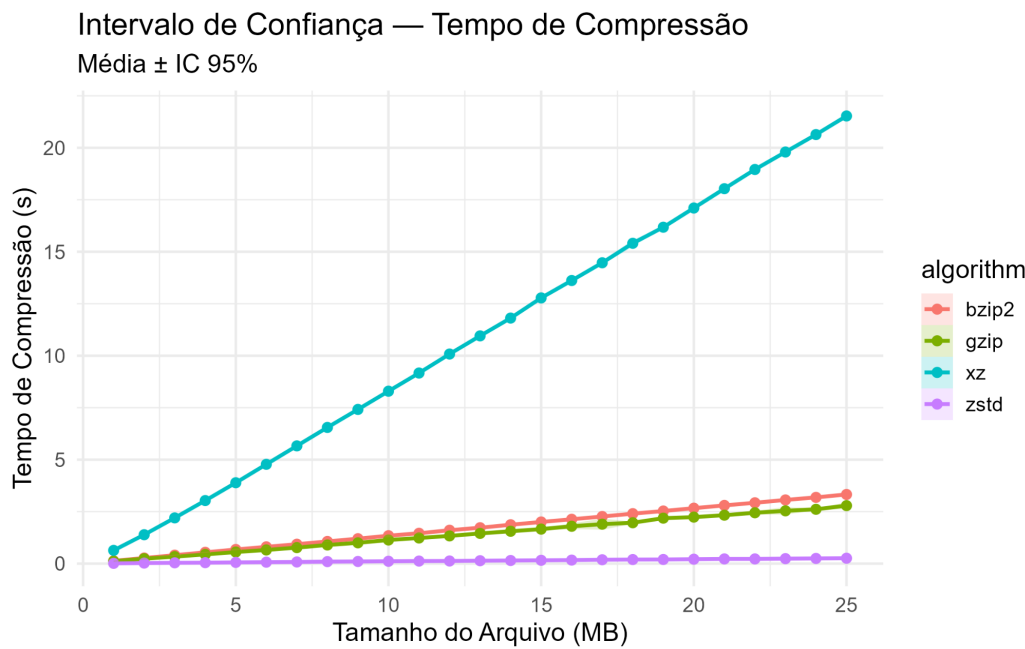


Fig. 10. Intervalo de confiança (95%) para tempo de compressão.

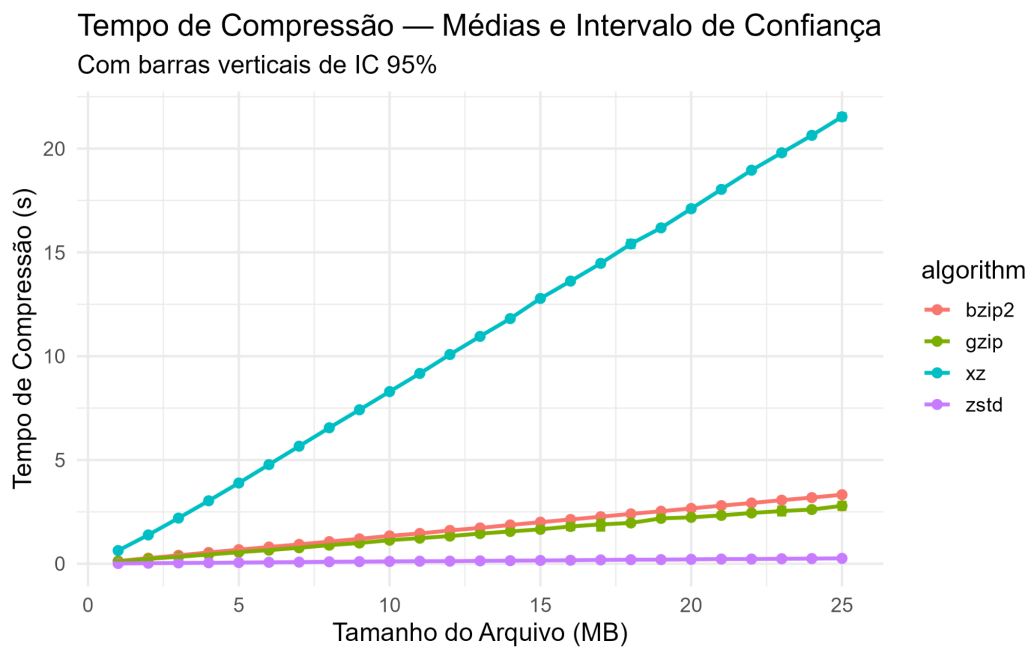


Fig. 11. Tempo de compressão com médias e intervalo de confiança de 95% (barras verticais).

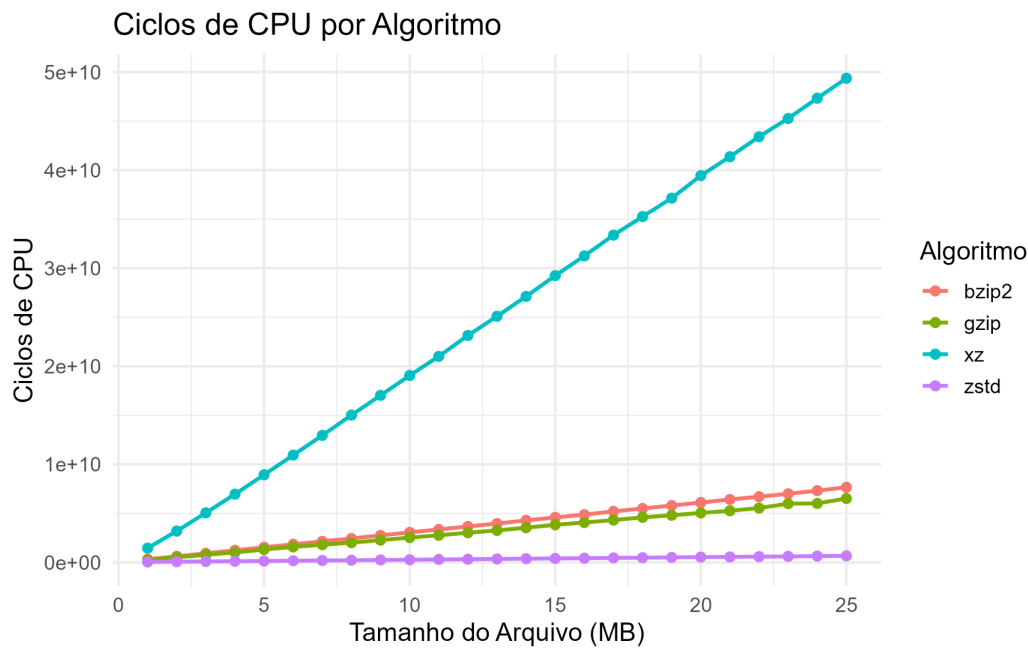


Fig. 12. Intervalo de confiança (95%) para tempo de descompressão.